Machine Learning Screening Tool for Efficient Mpox Diagnosis

Prepared by:
Sandeep Karmacharya MIT234062
Chimi Wangmo MIT233078
Elgiriyage Nisini Nethmila Silva MIT231399
Dewmini Aranayake MIT232203



Table of Contents

1. Introduction - Problem Statement and Background

Mpox (formerly known as monkeypox) is a disease caused by infection with the mpox virus that historically occurred mostly in tropical rainforest areas of Central and West Africa and has spread to other regions. Since May 2022, there has been a global increase in mpox infections in multiple countries where the illness is not usually seen with The World Health Organization (WHO) declaring the mpox outbreak a public health emergency of international concern on 23 July 2022 [1].

MPOX is a viral zoonotic disease. That means that it can spread between humans and animals and human to human on close contact making it contagious. In most people it takes 2-4 weeks for the symptoms to clear on its own once diagnosed [2]. However, the diagnosis of MPOX in itself is a challenge before treatment can begin. Tests for mpox are time-consuming because they need to be sent away to a laboratory for analysis. This can take days for people to find out their results [3]. On top of that MPOX tests are expensive. Most tests cost around £100, while private clinics charge around £200 for the tests. These factors highlight the need for a more efficient and cost-effective screening method.

The delay in receiving test results and the high costs associated with PCR testing create barriers to efficient disease management, especially in large populations or workforce settings where quick turnaround times are essential for such communicable diseases. Thus, there is a pressing need for an alternative method that can quickly and accurately screen individuals for MPOX, reducing the reliance on expensive and slow PCR tests.

This study explores whether machine learning can be utilized to develop a cost-effective screening tool for predicting MPOX infections without the need for a PCR test. Such a tool could potentially reduce the number of PCR tests required, thereby lowering costs and speeding up diagnosis within the workforce population.

2. Resources

2.1 Data Sources

The dataset contains information about clinical characteristics of monkeypox infection in humans during the 2022 outbreak at a central London centre. It can be found on Kaggle at https://www.kaggle.com/datasets/muhammad4hmed/monkeypox-patients-dataset

2.2 Characteristics of the data

This data includes 25,000 rows with 11 features. Among 11 features, rectal pain, sore throat, penile oedema, oral lesions, solitary lesion, swollen tonsils, HIV infection, sexually transmitted infection are Boolean data type and patient_id, systemic illness and MPOX PCR Result are object data type.

Table 1: Characteristics of monkeypox data

Attributes	Data Type
Patient_ID	Object
Systemic Illness	Object
Rectal Pain	Boolean
Sore Throat	Boolean
Penile Oedema	Boolean
Oral Lesions	Boolean
Solitary Lesion	Boolean
Swollen Tonsils	Boolean
HIV Infection	Boolean
Sexually Transmitted Infection	Boolean
MPOX PCR Result	Object

3. Business Model

3.1 Research Question

The main research question addressed in this study is, “Can machine learning be used to create a cost-effective screening tool for predicting individuals who may have contracted the MPOX virus without the need for a PCR test to reduce the number of PCR tests needed to be performed on the workforce population?”

3.2 Challenges during preprocessing and analysis

There were several challenges or limitation in this study. The main issue was the dataset having almost 25% as a missing value. It had to be removed which drastically, decreased the size of the dataset. With only 18784, the volume of the dataset can be considered as low because in machine learning, higher volume of the dataset is required for proper training and testing. The number of features in the dataset were also low with only 10 usable features which could have also been the main point for low accuracy. The dataset had objects and boolean as a datatype which was solved with preprocessing methods.

The study encountered challenges regarding privacy, ethical considerations, and data interpretation due to the inclusion of personal health information. Consequently, there were issues over the

processing, storage, and usage of the data. The columns labeled "Penile Oedema" and "Solitary Lesion" include confidential or private information, which presents difficulties in maintaining data privacy and complying with relevant regulations. Without expertise in the field, understanding the "Swollen Tonsils" and "Oral Lesions" categories can be challenging. This poses a difficulty in understanding the data properly for the analysis.

3.3 Pre-processing the data for analysis

3.3.1 Data Preparation

Step 1 – Download the csv file from the Kaggle website using this link:

https://www.kaggle.com/datasets/muhammad4hmed/monkeypox-patients-dataset

Step 2 – Load the dataset using pandas

```
[18]:
1 # Load the dataset
2 df = pd.read_csv('monkeypox.csv')
```

Step 3 – Check for missing values

```
[48]:
1 df.isnull().sum()
```

```
[48]: Patient_ID                        0
      Systemic Illness                6216
      Rectal Pain                        0
      Sore Throat                        0
      Penile Oedema                      0
      Oral Lesions                       0
      Solitary Lesion                    0
      Swollen Tonsils                    0
      HIV Infection                      0
      Sexually Transmitted Infection     0
      MonkeyPox                          0
      dtype: int64
```

Step 4 - Handling missing values

```
*[5]:
1 # Remove missing values
2 df.dropna(inplace=True)
```

The df.dropna syntax eliminates any rows that contain missing values, while setting inplace=True replaces the current dataframe with the new one.

Step 5 - Removing unnecessary columns

```
[7]:
1 df = df.drop('Patient_ID', axis = 1)
```

The Patient_ID was excluded to ensure anonymity in the analysis, as well as because this element is unnecessary for our analysis.

Step 6 - Convert categorical variables to numerical

The categorical data was transformed into numerical data using the LabelEncoder function from the sklearn module.

```
•[8]:
# Encode categorical variables to integar
label_encoders = {}
for column in df.select_dtypes(include=['object', 'bool']).columns:
    label_encoders[column] = LabelEncoder()
    df[column] = label_encoders[column].fit_transform(df[column])
```

```
[9]:
df.head()
```

[9]:

	Systemic Illness	Rectal Pain	Sore Throat	Penile Oedema	Oral Lesions	Solitary Lesion	Swollen Tonsils	HIV Infection	Sexually Transmitted Infection	MonkeyPox
1	0	1	0	1	1	0	0	1	0	1
2	0	0	1	1	0	0	0	1	0	1
4	2	1	1	1	0	0	1	1	0	1
5	2	0	1	0	0	0	0	0	0	0
6	0	0	1	0	0	0	0	1	0	1

3.4 Addressing the research question with the data set

To address our research question of developing a cost-effective and time-saving screening tool to predict individuals infected with MPOX virus, we utilized monkey data as follows.

Data Preprocessing

- Encoding the categorical symptom features (True/False values) into numerical values using techniques like label encoding.
- Splitting the data into training and testing sets in order to validate the performance of the model.

Model Training

- Training different supervised machine learning classification algorithms like Logistic Regression, Decision Trees, Random Forests, and Support Vector Machines with the training data set.
- Performing hyperparameter tuning using techniques like grid search cross-validation to optimize each model's performance on the training data.

Model Evaluation

- Evaluating the trained models using metrics such as accuracy, precision, recall, and F1-score. And choosing the best performance model as the proposed screening tool.
- Analysis of the importance of features to understand which symptoms are most predictive of a positive MPOX case according to the model.

Our goal is to implement an accurate cost-effective machine learning model that can identify individuals who may be positive for MPOX based on their reported symptoms. This model allows limited PCR testing resources to be prioritized for high-risk individuals predicted to be positive, reducing the overall number of PCR tests needed across the workforce population. Finally, the training model serves as a screening tool to improve operational efficiency and cost-effectiveness by effectively

identifying potential MPOX cases for confirmatory testing and treatment and minimizing unnecessary PCR testing.

3.5 Algorithms used in the project

For this study, various modules were used such as pandas for handling data, matplotlib and sns for visualization, sklearn for model selection, preprocessing and metrics, xgboost for gradient-boosting decision tree and warning modules to ignore the warnings to declutter the notebook.

[17]:
```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
import warnings

# Suppress warnings
warnings.filterwarnings('ignore')
```

The monkeypox dataset was trained using multiple machine learning algorithms. The machine learning models employed in this investigation include Random Forest (RF), Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Logistic Regression, Decision Tree (DT), Gradient Boosting, and XGBoost.

The brief explanation of each algorithm is as follow:

Random Forest: Random forest is a learning method that builds multiple decision trees and merge them to get more accurate and stable prediction. It uses a technique called bootstrap aggregation or bagging. Each decision tree uses random subset of data which provides features to consider when splitting nodes. This reduces risk of overfitting and increase model's robustness. However random forest can be computationally intensive and less interpretable than single decision tree.

Support Vector Machine (SVM): SVM is a learning algorithm used for classification and regression. SVM can use kernel functions to transform the data into a higher-dimensional space where a linear separator can be found. It finds the optimal hyperplane that best separates the classes in the feature space.

K-Nearest Neighbors (KNN): KNN is a non-parametric method used for classification and regression. It classifies a data point based on the majority class among its K-nearest neighbours in the feature space. This method is highly intuitive and simple and effective with a well-chosen K.

Logistic Regression: Logistic Regression is a statistical model used for binary classification. It models the probability that a given input belongs to a particular class using the logistic function. The logistic function outputs values between 0 and 1, which can be interpreted as probabilities. The model is trained using maximum likelihood estimation.

Decision Trees: It is a flowchart-like models that make predictions learning simple decision rules from data features.

Gradient Boosting: It is algorithm that sequentially adds decision trees with each new tree correcting errors made by previous one.

XGBoost: It is a optimized implementation of gradient boosting that is effective and fast.

To use these algorithms, the dataset was partitioned into training and testing datasets in an 80-20 ratio, with 80% allocated to the training data and 20% allocated to the testing data. The evaluate_model function was designed to assess the model's performance and display metrics such as accuracy, classification report, and confusion matrix.

[12]:
```python
# Function to train and evaluate models
def evaluate_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Accuracy: {accuracy}")
    print("Classification Report:\n", classification_report(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    return accuracy
```

The model was executed, and the accuracies of the model were displayed by utilizing a for loop to iterate through all the models.

[13]:
```python
# Initialize models
models = {
    'Random Forest': RandomForestClassifier(),
    'Support Vector Machine': SVC(),
    'K-Nearest Neighbors': KNeighborsClassifier(),
    'Logistic Regression': LogisticRegression(),
    'Decision Tree': DecisionTreeClassifier(),
    'Gradient Boosting': GradientBoostingClassifier(),
    'XGBoost': XGBClassifier()
}
```

*[14]:
```python
# Evaluate each model
model_accuracies = {}
for model_name, model in models.items():
    print(f"Evaluating {model_name}")
    accuracy = evaluate_model(model, X_train, y_train, X_test, y_test)
    model_accuracies[model_name] = accuracy

# Display model accuracies
print("Model Accuracies:", model_accuracies)
```

The models were optimized using hyperparameter tuning for each model using either RandomizedSearchCV or GridSearchCV from the sklearn library to identify the optimal parameters for each model.

[20]:

```python
# Hyperparameter tuning for Decision Tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for Grid Search
param_grid_dt = {
    'criterion': ['gini', 'entropy'],
    'splitter': ['best', 'random'],
    'max_depth': [None, 10, 20, 30, 40, 50],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize the model
dt_model = DecisionTreeClassifier(random_state=42)

# Perform Grid Search
grid_search_dt = GridSearchCV(estimator=dt_model, param_grid=param_grid_dt, cv=5, n_jobs=-1, verbose=2)
grid_search_dt.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters for Decision Tree: ", grid_search_dt.best_params_)

# Evaluate the best model with optimized hyperparameters
best_dt_model = grid_search_dt.best_estimator_
evaluate_model(best_dt_model, X_train, y_train, X_test, y_test)

```

For example, the provided code imports the DecisionTreeClassifier and GridSearchCV modules from the sklearn library in order to carry out hyperparameter optimization. The parameter grid dictionary is constructed with multiple options including gini or entropy for the criterion, best or random for the splitter, None, 10, 20, 30, 40, 50 for the maximum depth, 2, 5, 10 for the min_samples_split, and 1, 2, 4 for the min_samples_leaf. These parameters are utilized to generate several combinations of different parameters. GridSearchCV then utilizes these combinations to determine the best model with the optimal parameters. Once the optimal parameters have been identified, the best model is constructed, and its metrics are computed. This method was applied for all the machine learning models for comparison.

4. Data Analysis

4.1 Goals of this project

The goal is to develop a predictive model that can accurately identify potential MPOX cases based on clinical characteristics and symptoms, thereby reducing the number of PCR tests needed. This would not only lower the overall testing costs but also expedite the diagnostic process, enabling quicker isolation and treatment of infected individuals, and ultimately, helping to control the spread of the virus more effectively.

4.2 Data analytics tool and libraries

Apart from the machine learning algorithms we used in section 3.5 (Algorithms used in the project), we used the following libraries for data analysis and model building.

- pandas for handling data
- matplotlib and sns for visualization
- sklearn for model selection, preprocessing and metrics

```
In [1]: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
import warnings

# Suppress warnings
warnings.filterwarnings('ignore')
```

4.2.1 Pandas

We used read_csv() to load the monkeypox csv file to the data frame.

```
In [2]: # Load the dataset
df = pd.read_csv('monkeypox.csv')
```

We used head() to view the first 5 rows of the monkeypox data set.

```
In [3]: # Display the first few rows of the dataframe
df.head()
```

Out[3]:

	Patient_ID	Systemic Illness	Rectal Pain	Sore Throat	Penile Oedema	Oral Lesions	Solitary Lesion	Swollen Tonsils	HIV Infection	Sexually Transmitted Infection	MonkeyPox
0	P0	NaN	False	True	True	True	False	True	False	False	Negative
1	P1	Fever	True	False	True	True	False	False	True	False	Positive
2	P2	Fever	False	True	True	False	False	False	True	False	Positive
3	P3	NaN	True	False	False	False	True	True	True	False	Positive
4	P4	Swollen Lymph Nodes	True	True	True	False	False	True	True	False	Positive

To view the dimensions and summary of the DataFrame, we used shape() and info() methods respectively. We generated the descriptive statistics of the numerical columns in the DataFrame like count, mean, standard deviation, minimum, maximum, and quartile values by using the describe() method.

In [4]:
```
# Explore the dataset
print("Shape of the dataset:\n", df.shape)
print("Info of the dataset:\n", df.info())
print("Summary statistics:\n", df.describe())
print("Checking for missing values:\n", df.isnull().sum())
```

```
Shape of the dataset:
 (25000, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 11 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Patient_ID                      25000 non-null  object
 1   Systemic Illness                18784 non-null  object
 2   Rectal Pain                     25000 non-null  bool
 3   Sore Throat                     25000 non-null  bool
 4   Penile Oedema                   25000 non-null  bool
 5   Oral Lesions                    25000 non-null  bool
 6   Solitary Lesion                 25000 non-null  bool
 7   Swollen Tonsils                 25000 non-null  bool
 8   HIV Infection                   25000 non-null  bool
 9   Sexually Transmitted Infection  25000 non-null  bool
 10  MonkeyPox                       25000 non-null  object
dtypes: bool(8), object(3)
memory usage: 781.4+ KB
Info of the dataset:
 None
Summary statistics:

       Patient_ID Systemic Illness Rectal Pain Sore Throat Penile Oedema  \
count       25000            18784       25000       25000         25000
unique      25000                3           2           2             2
top            P0            Fever       False        True          True
freq            1             6382       12655       12554         12612

       Oral Lesions Solitary Lesion Swollen Tonsils HIV Infection  \
count         25000           25000           25000         25000
unique            2               2               2             2
top           False            True            True          True
freq          12514           12527           12533         12584

       Sexually Transmitted Infection MonkeyPox
count                           25000     25000
unique                              2         2
top                             False  Positive
freq                            12554     15909
Checking for missing values:
 Patient_ID                          0
Systemic Illness                  6216
Rectal Pain                          0
Sore Throat                          0
Penile Oedema                        0
Oral Lesions                         0
Solitary Lesion                      0
Swollen Tonsils                      0
HIV Infection                        0
Sexually Transmitted Infection       0
MonkeyPox                            0
dtype: int64
```

We used the astype() method to convert the Boolean data to integer values. The apply() method has been used to count the number of occurrences of each value in the Boolean column.

In [5]:
```
# Convert boolean columns to integers
bool_columns = ['Rectal Pain', 'Sore Throat', 'Penile Oedema', 'Oral Lesions', 'Solitary Lesion', 'Swollen Tonsils', 'HIV Infecti

# Convert boolean columns to integer for counting
df_encoded = df[bool_columns].astype(int)

# Count the occurrences of each value in boolean columns
bool_counts = df_encoded.apply(pd.Series.value_counts).fillna(0).T
```

The df.isnull().sum() method was used to count the number of missing values in each column.

In [48]:
```
df.isnull().sum()
```

Out[48]:
```
Patient_ID                          0
Systemic Illness                 6216
Rectal Pain                         0
Sore Throat                         0
Penile Oedema                       0
Oral Lesions                        0
Solitary Lesion                     0
Swollen Tonsils                     0
HIV Infection                       0
Sexually Transmitted Infection      0
MonkeyPox                           0
dtype: int64
```

We used dropna() to remove the missing values in the dataset.

```
In [24]: # Remove missing values
         df.dropna(inplace=True)
```

4.2.2 Matplotlib

We used this matplotlib library to visualize the bar graphs.

Visualizing the Frequency of Symptoms and Infection Status in Monkeypox Patients

Here we used the plt.subplots() function to create a figure and a grid of the axises. To rotate x-ticks for better readability, we used the xticks method. The plt.tight_layout() function has been used to adjust the spacing between plot elements to avoid overlapping. Finally, plt.show() used to inline the plot in a new window or Jupyter notebook.

```
# Create a figure and axes
fig, ax = plt.subplots(figsize=(12, 8))

# Plot the bar chart with the combined counts
bool_counts.plot(kind='bar', stacked=True, ax=ax, color=['#619CFF', '#F8766D'])

# Set labels and title
# ax.set_xlabel('Symptoms and Infection Status')
ax.set_ylabel('Count')
ax.set_title('Frequency of Symptoms and Infection Status in Monkeypox Patients')
ax.legend(title='Value', labels=['False', 'True'])

# Rotate x-ticks for better readability
plt.xticks(rotation=45, ha='right')

# Show the plot
plt.tight_layout()
plt.show()
```

Visualizing the Systemic Illness Distribution

To plot this chart, we used the same methods of visualizing the frequency of symptoms and infection status in monkeypox patients.

```
In [6]: # Create a figure and axes
        fig, ax = plt.subplots(figsize=(12, 3))

        # Count unique values in the 'Category' column
        value_counts = df['Systemic Illness'].value_counts()

        df_counts = pd.DataFrame([value_counts])

        # Plot the horizontal stacked bar chart
        df_counts.plot(kind='barh', stacked=True, ax=ax, color=['#619CFF', '#F8766D', '#00BA38'])

        # Add labels and title
        plt.xlabel('Systemic Illness Count')
        # plt.ylabel('Systemic Illness Count')
        plt.title('Systemic Illness Distribution')
        # Remove y-axis ticks and labels
        plt.tick_params(axis='y', which='both', left=False, labelleft=False)


        # Show the plot
        plt.show()
```

Visualizing the MonkeyPox Distribution

In [7]:
```
# Create a figure and axes
fig, ax = plt.subplots(figsize=(12, 3))

# Count unique values in the 'Category' column
value_counts = df['MonkeyPox'].value_counts()

df_counts = pd.DataFrame([value_counts])

# Plot the horizontal stacked bar chart
df_counts.plot(kind='barh', stacked=True, ax=ax, color=['#619CFF', '#F8766D'])

# Add labels and title
plt.xlabel('MonkeyPox Count')
# plt.ylabel('Systemic Illness Count')
plt.title('MonkeyPox Distribution')
# Remove y-axis ticks and labels
plt.tick_params(axis='y', which='both', left=False, labelleft=False)

# Show the plot
plt.show()
```

4.2.3 Sklearn for model selection

In order to split the dataset into training and testing sets, we imported train_test_split function from the sklearn.model_selection.

In [16]:
```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4.3 Methods and Visualization

We utilized a stacked bar plot to visualize the dataset since it effectively displays the distribution of values and provides a quick overview of the proportion of each category. As shown in figure 4, we can examine the frequencies of different categories for various attributes. At first look, it is evident that the distribution is normal as we have a about equal distribution of true and false values, with each feature comprising around 50% of the dataset. We have selected multiple machine learning models to gain a deeper comprehension of how each model predicts and assess their accuracies. Additionally, we have made efforts to enhance each model to identify the most effective model for this dataset as shown in table 3. We can observe the accuracy of each model and we observed highest performing algorithm was Gradient Boosting before hyperparameter optimization at 72% however, after hyperparameter optimisation SVM, Gradient Boosting and XGBoost performed equally at 72%.

5. Results

5.1 Descriptive analysis

Table 2: Summary of Monkey Pox dataset

	count	unique	top	freq
Patient_ID	25000	25000	P0	1
Systemic Illness	18784	3	Fever	6382
Rectal Pain	25000	2	FALSE	12655
Sore Throat	25000	2	TRUE	12554
Penile Oedema	25000	2	TRUE	12612
Oral Lesions	25000	2	FALSE	12514
Solitary Lesion	25000	2	TRUE	12527
Swollen Tonsils	25000	2	TRUE	12533
HIV Infection	25000	2	TRUE	12584
Sexually Transmitted Infection	25000	2	FALSE	12554
MonkeyPox	25000	2	Positive	15909

The table 2 above shows the descriptive statistics of the monkeypox dataset. It shows the counts for each features, unique values for each features, the most common values as top and its frequency.

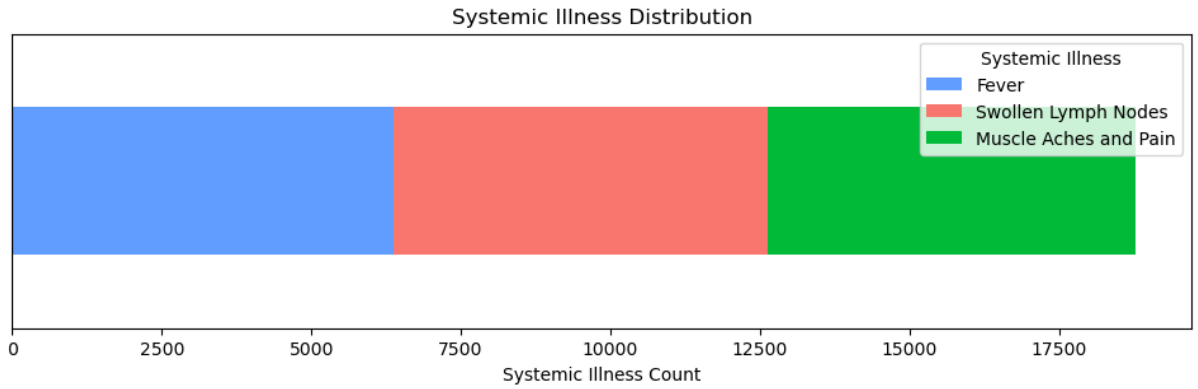

Figure 1: Bar graph of systemic illness distribution

This bar graph 1 illustrates the distribution of different illnesses among the patients in the dataset. Each bar represents a type of symptoms patients having during the study period. This visualization helps in understanding the prevalence of various systemic illnesses among the patients, highlighting which illnesses are most common.

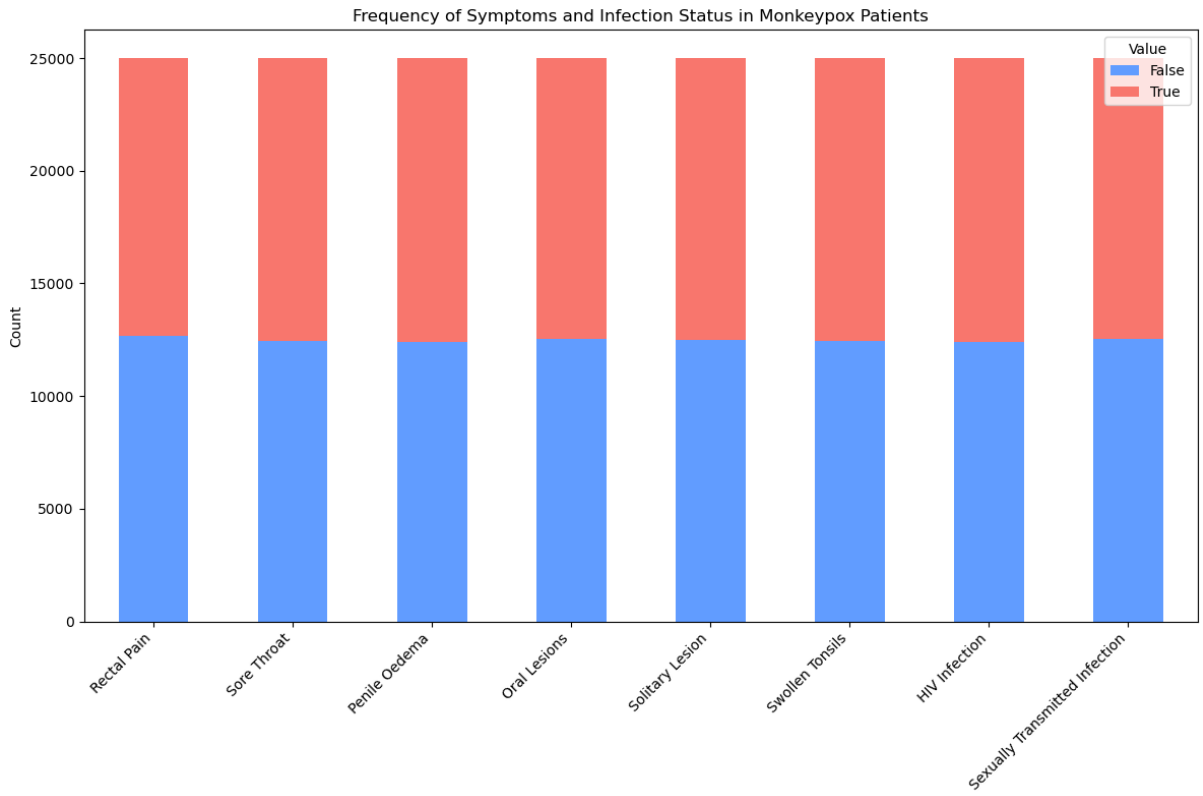

Figure 2: Bar graph of symptoms and infection distribution

The bar graph 2 illustrates the distribution of various symptoms (e.g., sore throat, penile oedema, solitary lesion) and their relationship with mpox infection. It shows how each symptom is common among the patients and provide useful information about which symptoms are most relevant of positive mpox diagnosis.

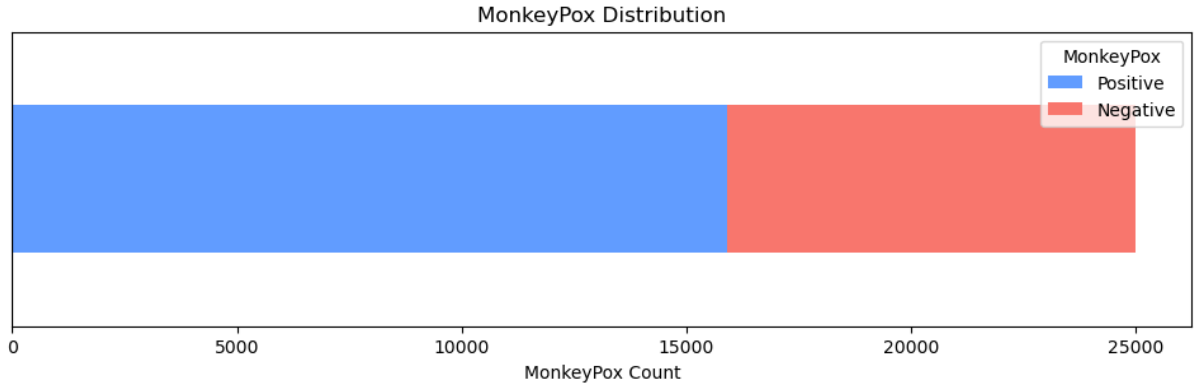

Figure 3: Bar graph of monkeypox distribution

This bar graph 3 shows the distribution of mpox infection results among patients. It helps to understand prevalence of mpox in the dataset and these results can be used to improve the validity of the predictive model.

5.2 Predictive analysis and hyperparameter optimization

Table 3: Comparison of different models before and after hyperparameter optimization

Model	Before hyper-parameter optimization					After hyper-parameter optimization				
	Precision		Recall		Accuracy	Precision		Recall		Accuracy
	0	1	0	1		0	1	0	1	
Random Forest	0.51	0.73	0.29	0.87	0.69	0.51	0.73	0.30	0.87	0.69
Support Vector Machine (SVM)	0.60	0.73	0.22	0.93	0.71	0.61	0.73	0.25	0.93	0.72
K-Nearest Neighbors	0.45	0.73	0.36	0.79	0.66	0.51	0.73	0.29	0.88	0.69
Logistic Regression	0.56	0.71	0.17	0.94	0.70	0.57	0.71	0.17	0.94	0.70
Decision Tree	0.50	0.73	0.30	0.86	0.67	0.50	0.73	0.30	0.86	0.69
Gradient Boosting	0.59	0.74	0.30	0.91	0.72	0.61	0.73	0.26	0.92	0.72
XGBoost	0.53	0.73	0.30	0.88	0.70	0.62	0.73	0.26	0.93	0.72

The table above displays the outcomes of various machine learning models which were utilized on the MonkeyPox dataset. The provided table displays the outcomes prior to and after hyperparameter optimization. Precision measures the accuracy of a model in correctly predicting values, while recall measures the comprehensiveness of positive predictions, indicating the ability to identify all relevant instances. Out of all the models used in this process, Gradient Boosting had higher accuracy of 72%. However, after adjusting hyperparameters, SVM, Gradient Boosting, and XGBoost demonstrated comparable levels of accuracy with 72%.

6. Discussions: Outcomes of the project

6.1 Impact of Preprocessing

Features	Original				Pre-processed			
	count	unique	top	freq	count	unique	top	freq
Patient_ID	25000	25000	P0	1	18784	18784	P1	1
Systemic Illness	18784	3	Fever	6382	18784	3	Fever	6382
Rectal Pain	25000	2	FALSE	12655	18784	2	FALSE	9514
Sore Throat	25000	2	TRUE	12554	18784	2	TRUE	9474
Penile Oedema	25000	2	TRUE	12612	18784	2	TRUE	9463

Oral Lesions	25000	2	FALSE	12514	18784	2	TRUE	9463
Solitary Lesion	25000	2	TRUE	12527	18784	2	TRUE	9423
Swollen Tonsils	25000	2	TRUE	12533	18784	2	TRUE	9419
HIV Infection	25000	2	TRUE	12584	18784	2	TRUE	9494
Sexually Transmitted Infection	25000	2	FALSE	12554	18784	2	FALSE	9465
MonkeyPox	25000	2	Positive	15909	18784	2	Positive	12585

The procedure of mitigating missing values had a significant influence on our study. The initial dataset contained 25,000 entries. However, after removing the missing values related to systemic illness, the number of rows in the dataset decreased to 18,784. This reduction represents a significant portion of the dataset, accounting for about 25%. The primary effect is observed on oral lesions. Prior to preprocessing, the majority of the data was determined to be false. However, following preprocessing, the majority of the data was altered to true. This may be the cause of reduced precision in our model. Utilizing various elimination methods, such as calculating the mean or median, could be a suitable approach to enhance accuracy.

7. Conclusions

The result of the study proves how models like SVM, Gradient Boosting and XGBoost can be trained to have high accuracy and effectively identify Mpox positive patients. The current study's model accuracy stands at 72% given the access limitation to large and diverse datasets to train the model accurately. However, the significance of the study is paramount. It has the potential clinical application to help in early diagnosis, patient stratification and treatment planning. Additionally, it can also help in drug discovery and development given that Mpox currently has no specific treatment but is still infectious with it spreading rapidly until the symptoms subside. As mild as it can be, owing to its infectious nature it will overwhelm the health industry if it becomes a pandemic and deprive those that need critical care of the services.

Thus, there is a need for more study and advancement by using large scale, more annotated dataset, further investigation of feature selection and dimensionality and addressing limitations addressed above to fully realize the model's accuracy in MPOX diagnosis and prognosis.

8. References

[1] A. G. D. of H. and A. Care, “Mpox (monkeypox).” Accessed: Jun. 04, 2024. [Online]. Available: https://www.health.gov.au/diseases/monkeypox-mpox

[2] “AI-Based Approaches for the Diagnosis of Mpox: Challenges and Future Prospects | Semantic Scholar.” Accessed: Jun. 04, 2024. [Online]. Available: https://www.semanticscholar.org/paper/AI-Based-Approaches-for-the-Diagnosis-of-Mpox%3A-and-Asif-Zhao/5db2be1e556b767dca22539561afda1d7c420a4a

[3] M.-S. Ong, F. Magrabi, and E. Coiera, “Delay in reviewing test results prolongs hospital length of stay: a retrospective cohort study,” *BMC Health Serv. Res.*, vol. 18, no. 1, p. 369, May 2018, doi: 10.1186/s12913-018-3181-z.

9. Appendix

9.1 Presentation Link

Video Link: https://youtu.be/Tyz60PiJ2Oc

9.2 Codes

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
import warnings

# Suppress warnings
warnings.filterwarnings('ignore')
```

```
# Load the dataset
df = pd.read_csv('monkeypox.csv')
# Display the first few rows of the dataframe
df.head()
# Explore the dataset
print("Shape of the dataset:\n", df.shape)
print("Info of the dataset:\n", df.info())
print("Summary statistics:\n", df.describe())
print("Checking for missing values:\n", df.isnull().sum())
# Convert boolean columns to integers
bool_columns = ['Rectal Pain', 'Sore Throat', 'Penile Oedema', 'Oral
Lesions', 'Solitary Lesion', 'Swollen Tonsils', 'HIV Infection', 'Sexually
Transmitted Infection']

# Convert boolean columns to integer for counting
df_encoded = df[bool_columns].astype(int)

# Count the occurrences of each value in boolean columns
bool_counts = df_encoded.apply(pd.Series.value_counts).fillna(0).T

# Combine the counts
# symptom_counts = pd.concat([categorical_counts, bool_counts], axis=0)

# Create a figure and axes
fig, ax = plt.subplots(figsize=(12, 8))

# Plot the bar chart with the combined counts
bool_counts.plot(kind='bar', stacked=True, ax=ax, color=['#619CFF',
'#F8766D'])

# Set labels and title
# ax.set_xlabel('Symptoms and Infection Status')
ax.set_ylabel('Count')
ax.set_title('Frequency of Symptoms and Infection Status in Monkeypox
Patients')
ax.legend(title='Value', labels=['False', 'True'])
```

```
# Rotate x-ticks for better readability
plt.xticks(rotation=45, ha='right')

# Show the plot
plt.tight_layout()
plt.show()
# Create a figure and axes
fig, ax = plt.subplots(figsize=(12, 3))

# Count unique values in the 'Category' column
value_counts = df['Systemic Illness'].value_counts()

df_counts = pd.DataFrame([value_counts])

# Plot the horizontal stacked bar chart
df_counts.plot(kind='barh', stacked=True, ax=ax, color=['#619CFF',
'#F8766D', '#00BA38'])

# Add labels and title
plt.xlabel('Systemic Illness Count')
# plt.ylabel('Systemic Illness Count')
plt.title('Systemic Illness Distribution')
# Remove y-axis ticks and labels
plt.tick_params(axis='y', which='both', left=False, labelleft=False)



# Show the plot
plt.show()
# Create a figure and axes
fig, ax = plt.subplots(figsize=(12, 3))

# Count unique values in the 'Category' column
value_counts = df['MonkeyPox'].value_counts()

df_counts = pd.DataFrame([value_counts])
```

```
# Plot the horizontal stacked bar chart
df_counts.plot(kind='barh', stacked=True, ax=ax, color=['#619CFF',
'#F8766D'])

# Add labels and title
plt.xlabel('MonkeyPox Count')
# plt.ylabel('Systemic Illness Count')
plt.title('MonkeyPox Distribution')
# Remove y-axis ticks and labels
plt.tick_params(axis='y', which='both', left=False, labelleft=False)

# Show the plot
plt.show()
df.isnull().sum()
# Remove missing values
df.dropna(inplace=True)
df.shape
df.describe()
df = df.drop('Patient_ID', axis = 1)
# Encode categorical variables to integar
label_encoders = {}
for column in df.select_dtypes(include=['object', 'bool']).columns:
    label_encoders[column] = LabelEncoder()
    df[column] = label_encoders[column].fit_transform(df[column])
df.head()
X = df.drop('MonkeyPox', axis = 1)
y = df['MonkeyPox']
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Function to train and evaluate models
def evaluate_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
```

```
    print(f"Accuracy: {accuracy}")
    print("Classification Report:\n", classification_report(y_test,
y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    return accuracy
# Initialize models
models = {
    'Random Forest': RandomForestClassifier(),
    'Support Vector Machine': SVC(),
    'K-Nearest Neighbors': KNeighborsClassifier(),
    'Logistic Regression': LogisticRegression(),
    'Decision Tree': DecisionTreeClassifier(),
    'Gradient Boosting': GradientBoostingClassifier(),
    'XGBoost': XGBClassifier()
}
# Evaluate each model
model_accuracies = {}
for model_name, model in models.items():
    print(f"Evaluating {model_name}")
    accuracy = evaluate_model(model, X_train, y_train, X_test, y_test)
    model_accuracies[model_name] = accuracy
# Display model accuracies
print("Model Accuracies:", model_accuracies)
# Hyperparameter tuning for Random Forest
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

# Define the parameter grid for Randomized Search
param_dist_rf = {
    'n_estimators': [50, 100, 200, 500],
    'max_features': ['auto', 'sqrt', 'log2'],
    'max_depth': [None, 10, 20, 30, 40, 50],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}
```

```
# Initialize the model
rf_model = RandomForestClassifier(random_state=42)

# Perform Randomized Search
random_search_rf = RandomizedSearchCV(estimator=rf_model,
param_distributions=param_dist_rf, n_iter=100, cv=5, n_jobs=-1, verbose=2,
random_state=42)
random_search_rf.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters for Random Forest: ",
random_search_rf.best_params_)

# Evaluate the best model with optimized hyperparameters
best_rf_model = random_search_rf.best_estimator_
evaluate_model(best_rf_model, X_train, y_train, X_test, y_test)
# Hyperparameter tuning for KNN
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for Grid Search
param_grid_knn = {
    'n_neighbors': [3, 5, 7, 9, 11, 13, 15, 17, 19, 21],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

# Initialize the model
knn_model = KNeighborsClassifier()

# Perform Grid Search
grid_search_knn = GridSearchCV(estimator=knn_model,
param_grid=param_grid_knn, cv=5, n_jobs=-1, verbose=2)
grid_search_knn.fit(X_train, y_train)
```

```
# Display the best parameters
print("Best Parameters for KNN: ", grid_search_knn.best_params_)

# Evaluate the best model with optimized hyperparameters
best_knn_model = grid_search_knn.best_estimator_
evaluate_model(best_knn_model, X_train, y_train, X_test, y_test)
# Hyperparameter tuning for Logistic Regression
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for Grid Search
param_grid_lr = {
    'C': [0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2', 'elasticnet', 'none'],
    'solver': ['lbfgs', 'saga', 'liblinear', 'sag'],
    'max_iter': [100, 200, 500]
}

# Initialize the model
lr_model = LogisticRegression(random_state=42)

# Perform Grid Search
grid_search_lr = GridSearchCV(estimator=lr_model,
param_grid=param_grid_lr, cv=5, n_jobs=-1, verbose=2)
grid_search_lr.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters for Logistic Regression: ",
grid_search_lr.best_params_)

# Evaluate the best model with optimized hyperparameters
best_lr_model = grid_search_lr.best_estimator_
evaluate_model(best_lr_model, X_train, y_train, X_test, y_test)
# Hyperparameter tuning for Decision Tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
```

```
# Define the parameter grid for Grid Search
param_grid_dt = {
    'criterion': ['gini', 'entropy'],
    'splitter': ['best', 'random'],
    'max_depth': [None, 10, 20, 30, 40, 50],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize the model
dt_model = DecisionTreeClassifier(random_state=42)

# Perform Grid Search
grid_search_dt = GridSearchCV(estimator=dt_model,
param_grid=param_grid_dt, cv=5, n_jobs=-1, verbose=2)
grid_search_dt.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters for Decision Tree: ", grid_search_dt.best_params_)

# Evaluate the best model with optimized hyperparameters
best_dt_model = grid_search_dt.best_estimator_
evaluate_model(best_dt_model, X_train, y_train, X_test, y_test)
# Initialize Gradient Boosting Classifier
gb_model = GradientBoostingClassifier(random_state=42)

# Define hyperparameters for tuning
param_grid_gb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5],
    'subsample': [0.7, 0.8, 0.9]
}

# Perform Grid Search
```

```
grid_search_gb = GridSearchCV(estimator=gb_model,
param_grid=param_grid_gb, cv=5, n_jobs=-1, verbose=2)
grid_search_gb.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters for Gradient Boosting: ",
grid_search_gb.best_params_)

# Evaluate the best model with optimized hyperparameters
best_gb_model = grid_search_gb.best_estimator_
evaluate_model(best_gb_model, X_train, y_train, X_test, y_test)
# Initialize XGBoost Classifier
xgb_model = XGBClassifier(random_state=42, use_label_encoder=False,
eval_metric='logloss')

# Define hyperparameters for tuning
param_grid_xgb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9]
}

# Perform Grid Search
grid_search_xgb = GridSearchCV(estimator=xgb_model,
param_grid=param_grid_xgb, cv=5, n_jobs=-1, verbose=2)
grid_search_xgb.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters for XGBoost: ", grid_search_xgb.best_params_)

# Evaluate the best model with optimized hyperparameters
best_xgb_model = grid_search_xgb.best_estimator_
evaluate_model(best_xgb_model, X_train, y_train, X_test, y_test)
```